David Arthur

Apache Kafka PMC


kafka

Gradle TECHNOLOGIES

WEBINAR

# Lessons learned modernizing Apache Kafka CI builds with Develocity and GitHub Actions

# Intro

- Long-time user of Apache Kafka ® (since before Apache days)
- Committer since 2014, PMC since 2023
- Wrote the original Gradle build for Kafka
- Worked on KRaft and the migration away from Apache ZooKeeper
- Working at Confluent for the past 7 years

# Part 1
# Build Modernization

# Project Info

- Large number of contributors (over 1000 across project life time)
- Relatively high rate of change (30-40 commits per week)
- Large test suite (34k JUnit tests)
- Multiple build targets (JVM and Scala versions)
- One build per commit (on trunk)

*168 hours/week ÷ 4 hours/build ≈ 40 builds per week (max)*

# Our Jenkins Experience

Prior to August of 2024, Apache Kafka exclusively used the Jenkins environment managed by ASF Infra. We had quite a few problems with this build over the years.

- Inconsistent run times
- Flakiness
- Hard to debug
- Long timeouts
- Noisy neighbors
- Green/blue builds were rare

# Re-Assessing Our Needs

**Stability**: A build should run to completion without errors or anomalies from the host system. Build failures should be test failures, not system failures

**Predictable**: Build runtimes should be consistent. A developer seeing a build in progress should have a rough idea when it will be finished.

**Scalability**: The system should allow a large number of parallel builds for PRs without queuing, resource contention, or other side effects.

# Caching to the Rescue (?)

Maybe we can solve this using a build cache?

- Make changes to kafka/metadata/src/main/…
- Run ./gradlew --build-cache test
- Only run the affected code in kafka/metadata
- Profit!

Alas, persistence between builds in Jenkins is tricky. Plugins require blessing from Infra. Maybe GitHub Actions can help here…

# GitHub Actions

- Workflow automation for GitHub repos
- Very generalized, YAML configurable
- Lots of built-in features
- Even more first-class community provided features
- Native GitHub feature with nice UI and excellent PR integration

# A Plan Emerges

- Develop a minimal GitHub Actions workflow
- Enable the Gradle build cache
- Collect data on builds
- Run both builds for 30 days
- Get feedback from community

# Develocity Enters the Chat

# Develocity

We want to quantify the difference between GHA and Jenkins. Gathering data manually from builds is *hard*. Luckily, we discovered that Develocity was already freely available for ASF projects. To help organize the builds, we introduced some tags.

- jenkins
- github
- trunk
- local (builds on laptops)

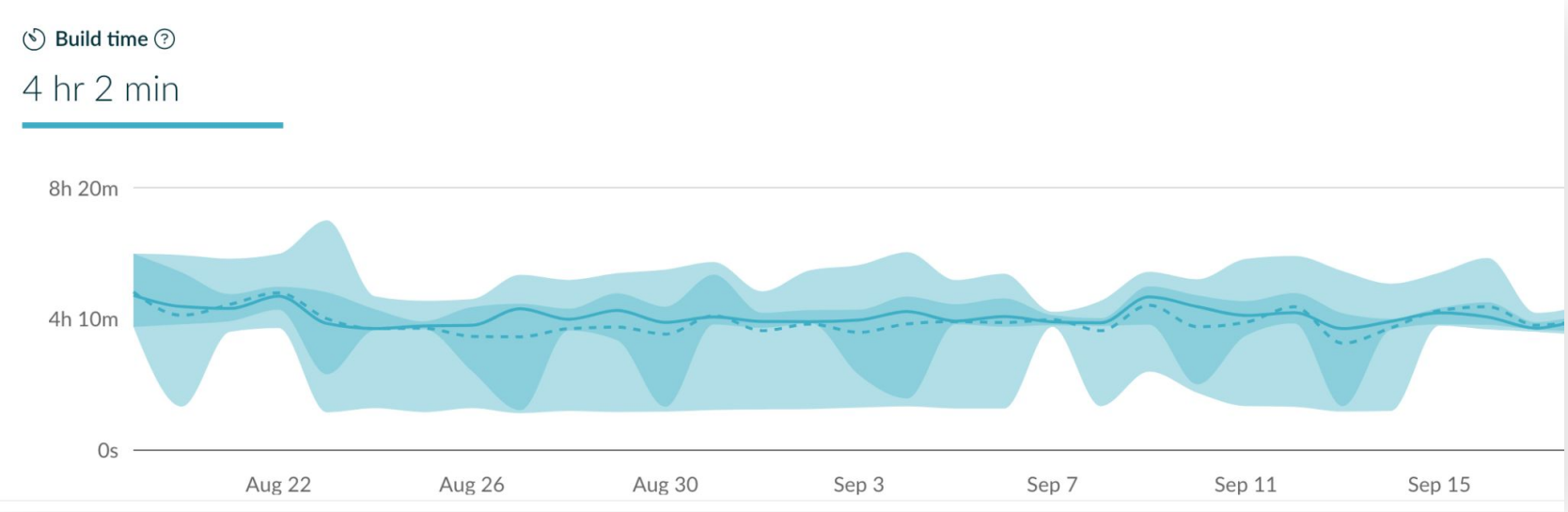
*Examples, "tags=jenkins,trunk" or "tags=github,not:trunk"*

# Lesson 1 – Build Scans are Awesome!

Developers love observability on their PRs. Focusing in on test failures without combing through stdout or downloading logs is a huge win.

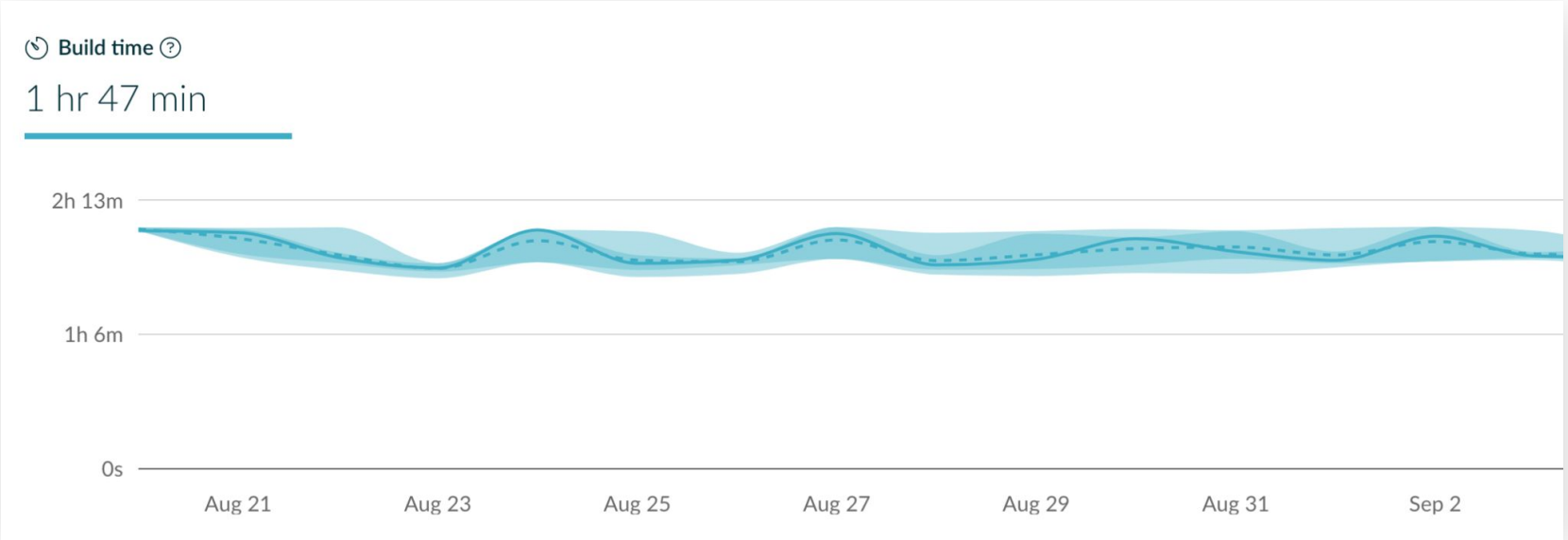
For folks who work on builds, there are many useful tools in Develocity. The performance trends, test trends, individual build scan metrics, A/B build comparisons. It's really quite great!

# Early Results – Jenkins




5%: 1h19m 50%: 4h55m 95%: 5h42m

# Early Results – GHA




5%: 1h37m 50%: 1h45m 95%: 1h59m

# Huh?

# Early Results – Analysis

Jenkins has a lot more resources than than the standard 4 core runners on GitHub Actions. Why is GitHub Actions seemingly so much faster than Jenkins?

In the best case on Jenkins, we had build times a bit over 1 hour. In the best case on GHA, these times are about 30-40% slower.

What is going on? Why is the average build time so much better on GHA?

# Lesson 2 – Resource Isolation

GitHub Actions runners are docker containers running on Azure VMs. This gives far superior resource isolation compared to a non-containerized Jenkins deployment

Isolation leads to stability. Stability leads to consistent and repeatable builds.

Simply moving the execution of our build from Jenkins to GHA instantly made our build more consistent and predictable.

# Build Cache – Idea

The basic strategy here is to populate a cache with trunk builds, and leverage the cache on PR builds.

The hope is that if your PR is mostly up-to-date with trunk, then there will be mostly cache hits and only the changed code will be rebuilt/retested

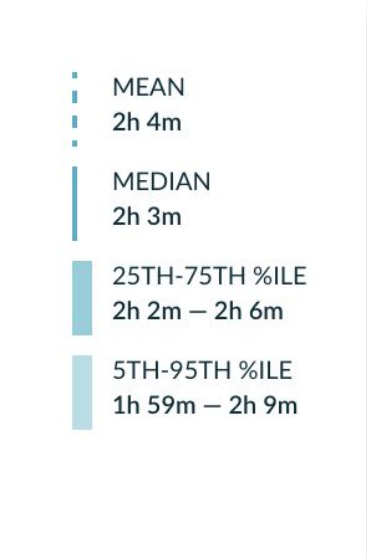
# Build Cache – Pitfalls

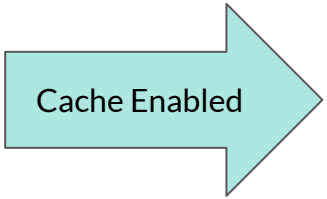
There were quite a few papercuts when getting caching set up

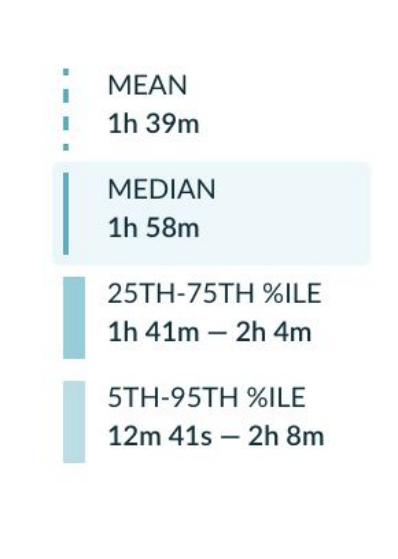
- Cache writer and cache reader jobs need to use the same workflow name
- Easy to bust the cache with timestamps and Git SHAs
- Overall, kind of tedious to debug when it isn't working (Develocity helps)

*Pro Tip: run with* ***GRADLE_BUILD_ACTION_CACHE_DEBUG_ENABLED=1***

# Build Cache – Results

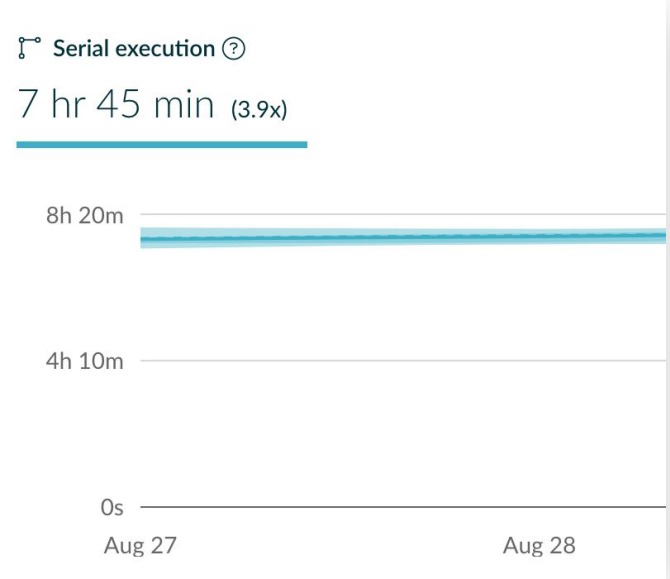


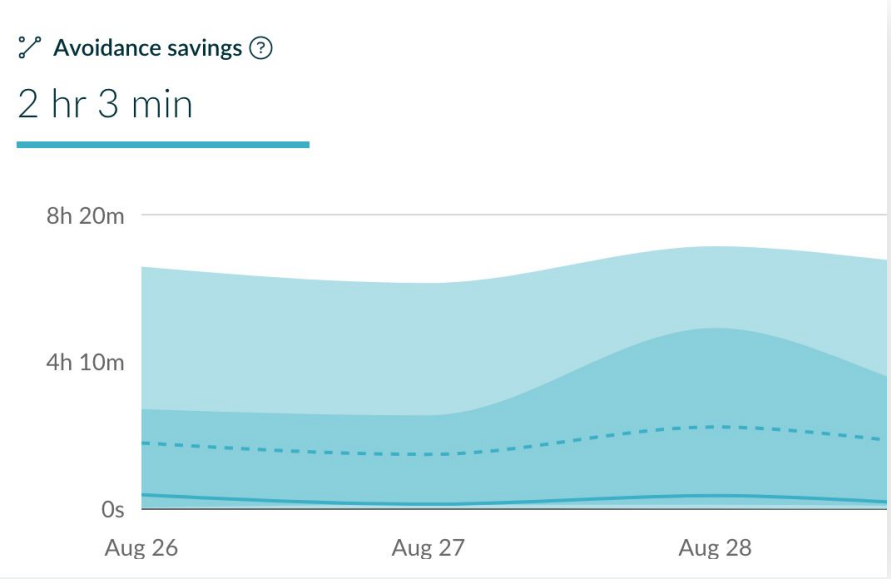



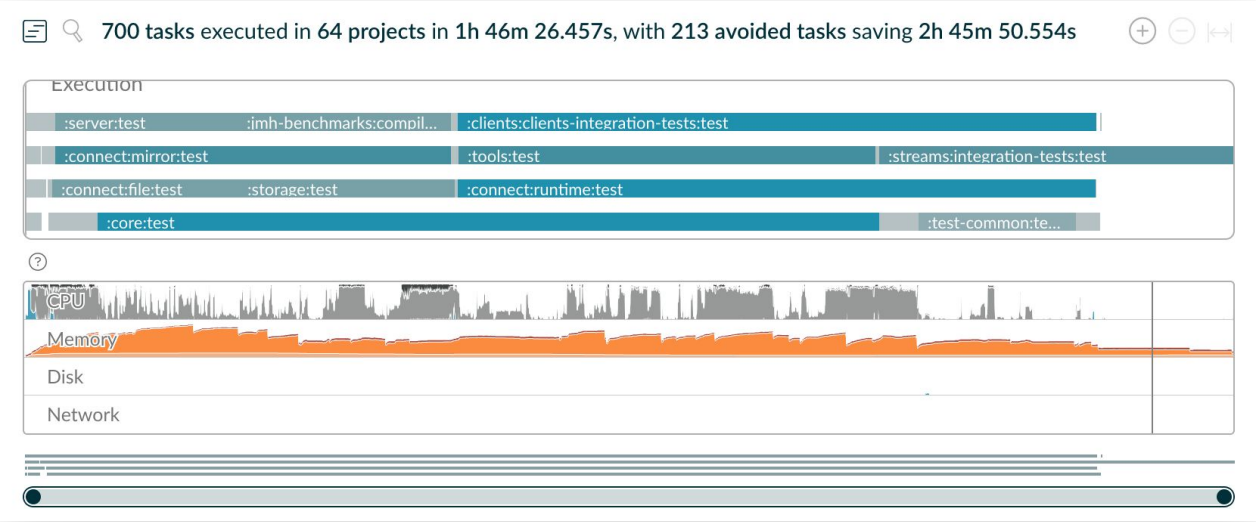

These days, on average we save around 15% of build time on PRs. In the best cases, we save 75-80%.

# Build Cache – Serial vs Wall Time



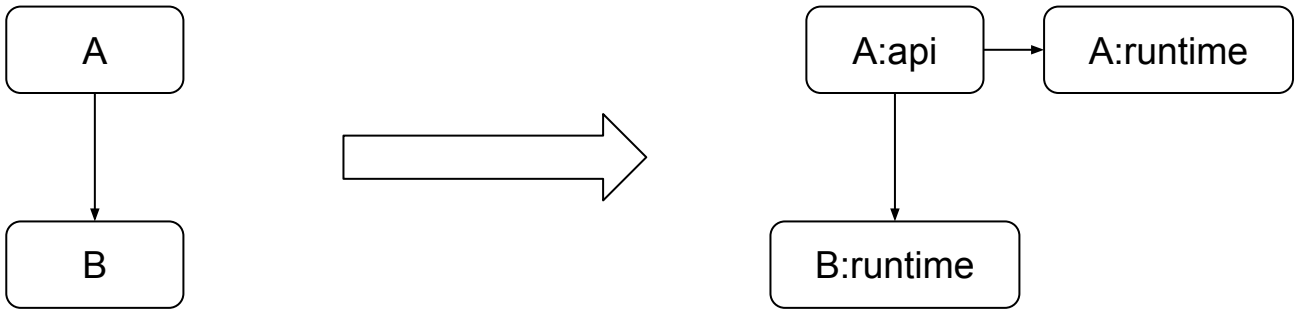
# Build Cache – Not Magic.




Looking at a typical PR build, we can see one of the four cores runs longer than the others. This is due to overly coarse Gradle test targets.

# Build Cache – Going Further

The next step for improving our cache hits will require refactoring the code within Kafka. By splitting our modules into "api" and "runtime" modules, we can greatly reduce the ripple effect of changes near the top of the dependency tree



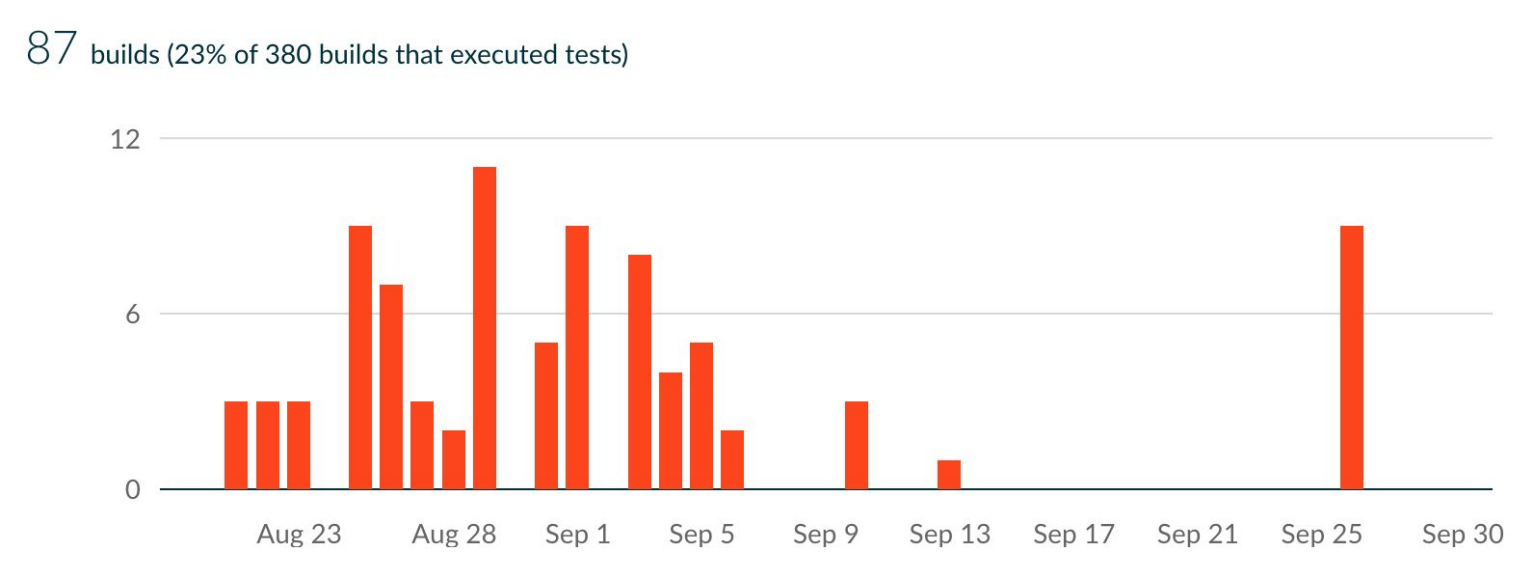
# Lesson 3 – Code Layout is Important

Smaller modules with minimal inter-dependencies improve the "bin packing" of tasks onto the Gradle workers. Splitting modules into "api" and "runtime" can also help minimize the impact of changes and improve build cache usage.

Also, it's good code design!

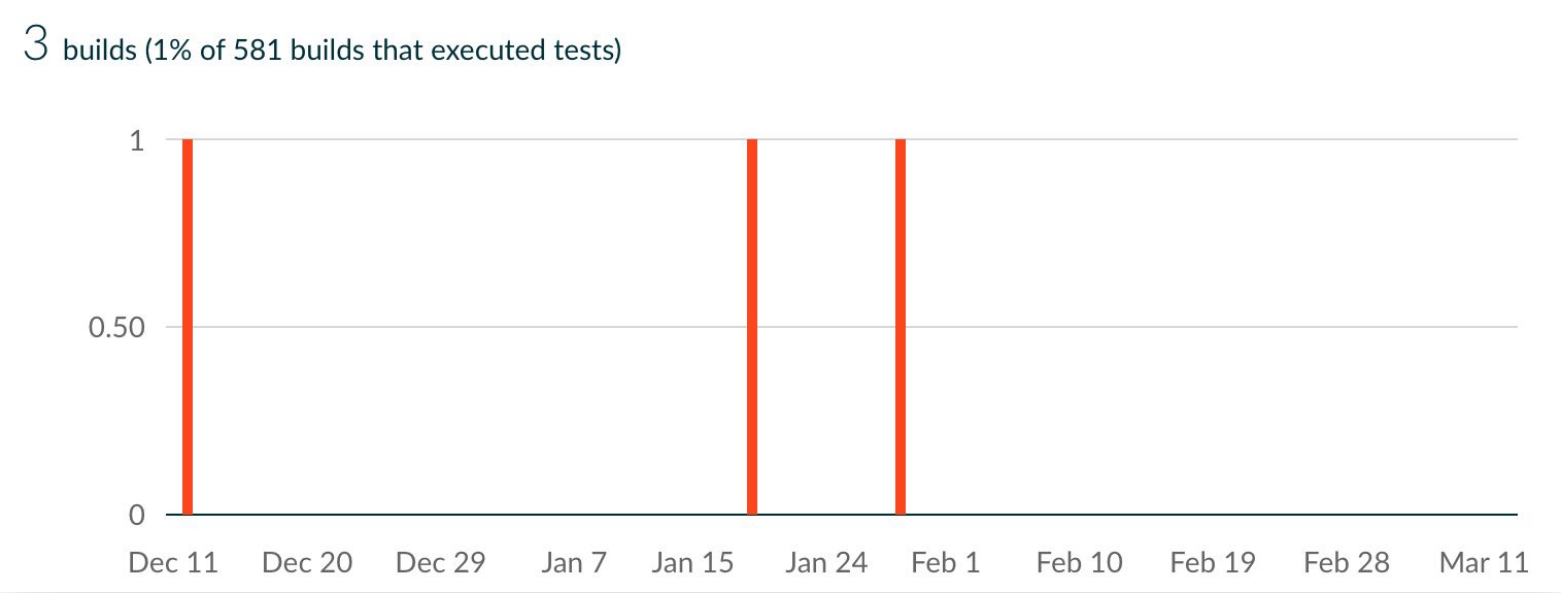
# Part 2
# Flaky Tests

# Flaky Tests – We had plenty




Failed builds prior to de-flaking effort (Aug/Sept 2024)

# Flaky Tests – Still have 'em 😓




Failed builds due to flaky tests in last 90 days

# What is a Flaky Test?

A test which, for identical code, sometimes passes and sometimes fails.

But Why?

- Non-determinism in the code
- Wall time (see above)
- Concurrency (threads and cores)
- Environment (disks, network, kubernetes, etc)
- Cosmic rays
- Gremlins in the build machine

# Flaky Tests – Retries

A simple retry policy can really help. Develocity has a Gradle plugin to allow individual tests to be retried.

```
develocity {
    testRetry {
        maxRetries = 1
        maxFailures = 10
    }
}
```

Automatic retires should be preferred to manual retries (i.e., clicking a "re-run" button somewhere).

# Flaky Tests – Reporting

With a retry policy in place, hopefully your build can be green more often than red. After several days collecting data, you can identify the most flaky tests in your project.

Test classes by flaky count

| Name | Outcome trend | Failed | Flaky |
|---|---|---|---|
| org.apache.kafka.tools.streams.DescribeStreamsGroupTest | | 0 (0%) | 13 (24%) |
| kafka.api.PlaintextConsumerTest | | 0 (0%) | 13 (23%) |
| org.apache.kafka.storage.internals.log.RemoteIndexCacheTest | | 0 (0%) | 13 (12%) |
| kafka.api.AuthorizerIntegrationTest | | 0 (0%) | 6 (11%) |
| kafka.server.GssapiAuthenticationTest | | 0 (0%) | 5 (9%) |

# Flaky Tests – File, Disable, Fix

We have found that an aggressive "disable and fix" policy has worked the best. Use annotations in the code to disable tests which need to be "deflaked"

```
@Timeout(60)
class PlaintextConsumerTest extends AbstractConsumerTest {
```

This disables and documents the flaky test

```
  @Flaky("KAFKA-18031")
  @ParameterizedTest(name = TestInfoUtils.TestWithParameterizedGrou
  @MethodSource(Array("getTestGroupProtocolParametersAll"))
  def testCloseLeavesGroupOnInterrupt(groupProtocol: String): Unit
    val adminClient = createAdminClient()
    val consumer = createConsumer()
    val listener = new TestConsumerReassignmentListener()
    consumer.subscribe(java.util.List.of(topic), listener)
    awaitRebalance(consumer, listener)
```

# Flaky Tests – Quarantine 😷

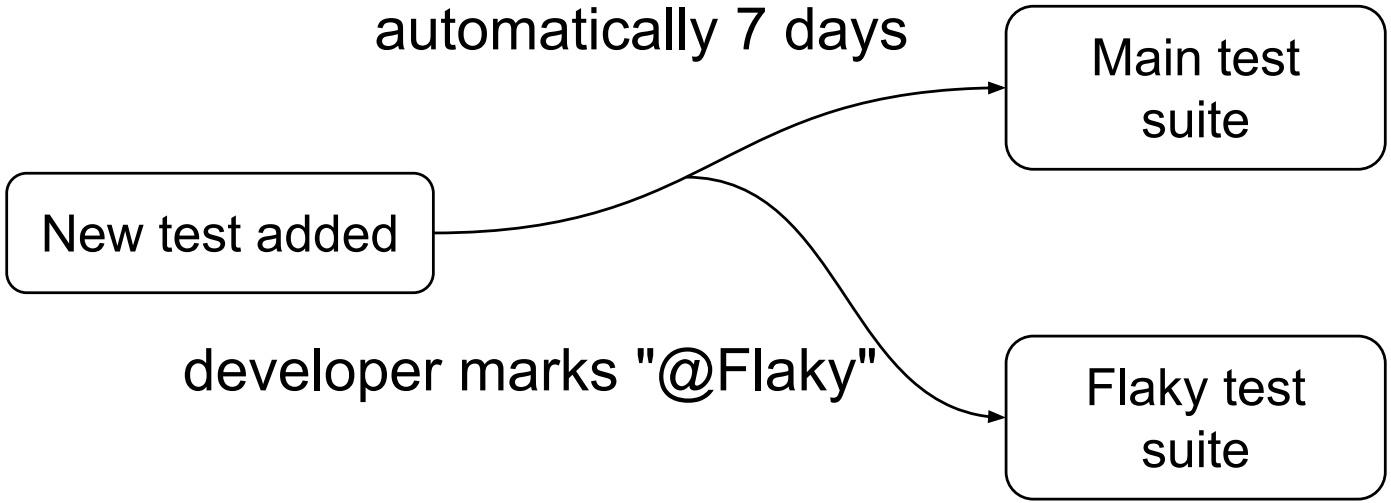
Instead of totally disabling flaky tests, we put them into a quarantine. This is a separate job that runs only the flaky tests and has a more permissive retry policy (3 retries for each test).

```
develocity {
    testRetry {
        maxRetries = 3
        maxFailures = 10
    }
}
```

We do this ensure adequate data is generated to verify flaky fixes. Only re-enable a fixed flaky test after there is sufficient evidence!

# Flaky Tests – An Ounce of Prevention

Newly committed tests are placed into the quarantine for 7 days. This gives us a chance to find flaky tests before they can start impacting builds.



# Lesson 4 – It takes a Village

No "Broken Windows". If your build fails regularly, developers will learn to ignore failures and neglect things further. Keeping the mainline build stable is critical.

Have an explicit policy written down. Anyone should be able to follow the flaky test policy, not just those "in the know".

Minimize developer toil. When we click "Rerun" several times per day, we stop thinking or caring as to why things are failing.

Blameless culture. Not "hey, you broke the build", but "hey, this test is failing – can you help?"

# Next Steps for Apache Kafka

- Leverage Develocity's Remote Build Cache
- Use GitHub Merge Queue
- Refactor modules to be more cache-friendly
- Bigger GHA runners for faster builds

Contributions welcome!!

# Thanks!

Special thanks to the wonderful ASF Infra team, Gradle Inc, and the many Apache Kafka committers who reviewed the build changes and provided feedback.

- GitHub: mumrah
- LinkedIn: davidrarthur
- Substack: Building Apache Kafka